Capture Your First Packet with tcpdump

Background

Network traffic analysis helps security analysts understand how systems communicate over a network. Packet capture tools can collect network traffic and display information such as source and destination addresses, protocols, ports, and packet contents.

`tcpdump` is a command-line network analysis tool used to capture, filter, and inspect packets. It can also save captured traffic to a PCAP file for later analysis.

Activity Overview

In this lab, I used `tcpdump` in a Linux terminal to identify available network interfaces, inspect live network traffic, capture HTTP-related packets, and examine the saved packet data.

I also explored how command-line options can control the number of captured packets, display detailed packet information, disable name resolution, and display packet contents in hexadecimal and ASCII formats.

Objective

- Identify available network interfaces.
- Inspect live network traffic using `tcpdump`.
- Capture selected network packets into a PCAP file.
- Read and examine captured packet headers.
- Inspect hexadecimal and ASCII packet data.
- Understand common `tcpdump` options and their purposes.

Tools Used

- **Linux terminal:** Used to execute network analysis commands.
- **tcpdump:** Used to capture, filter, and inspect network packets.
- **curl:** Used to generate HTTP-related network traffic.
- **PCAP file:** Used to save captured packets for later analysis.

Lab Environment

The lab was performed in the Linux terminal provided by the course. The `eth0` interface was used for the packet capture exercises, and the captured data was saved in a file named `capture.pcap`.

Task 1: Identify Network Interfaces

I used the `ifconfig` command to inspect the network interfaces available in the Linux environment.

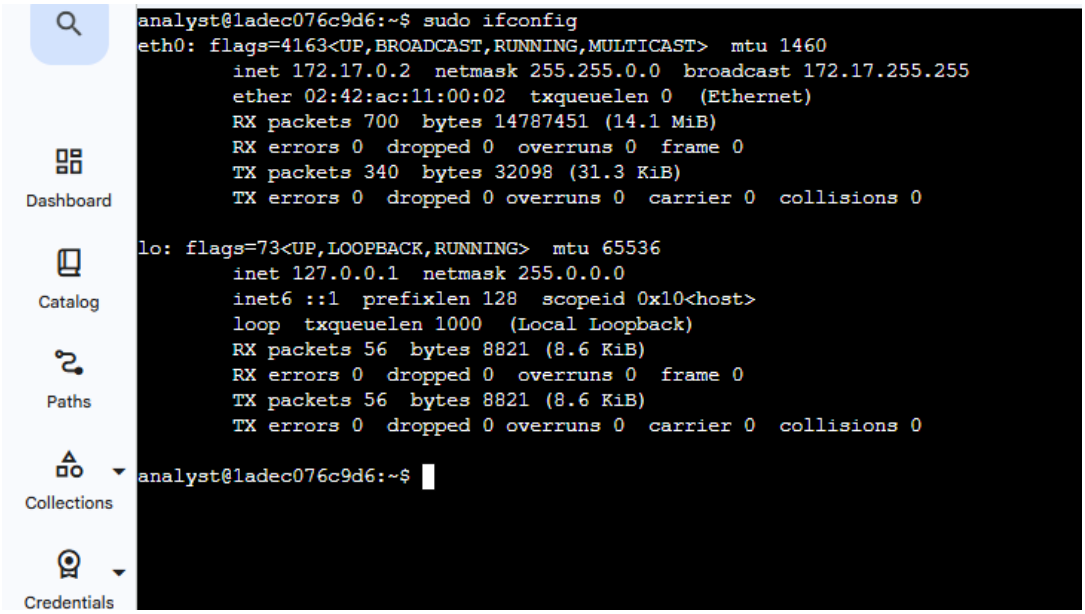
Command

```
sudo ifconfig
```

The command displays network interface information, including IP addresses, MAC addresses, interface status, and packet statistics.

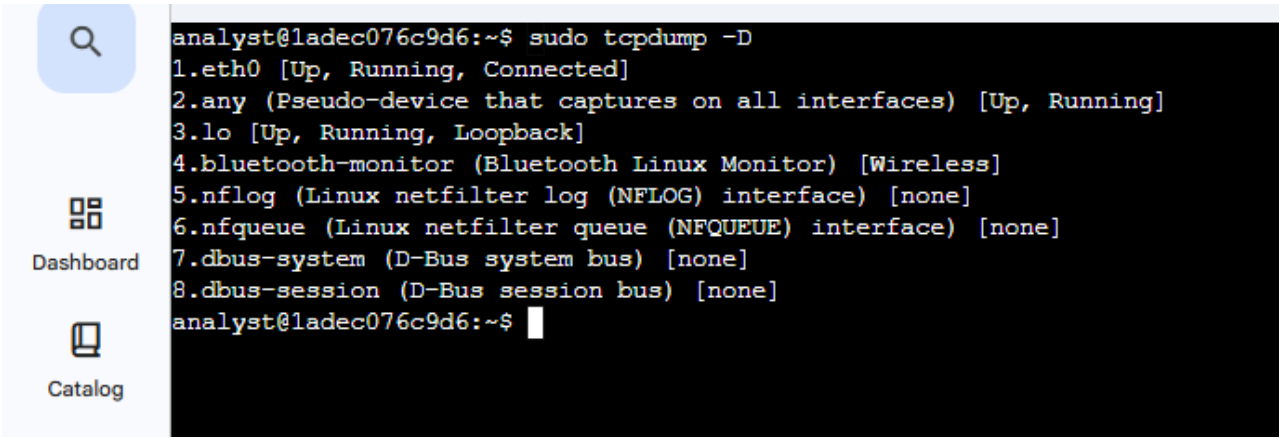
The lab identified `eth0` as the Ethernet interface used for the following capture activities. The `lo` interface represents the local loopback interface.

I also used `tcpdump` to list the interfaces available for packet capture.



Command

```
sudo tcpdump -D
```



Key learning: Identifying the correct network interface is an important first step because packet capture depends on which interface is monitored.

Task 2: Inspect Live Network Traffic

I used `tcpdump` to inspect a limited number of packets from the `eth0` interface.

Command

```
sudo tcpdump -i eth0 -v -c5
```

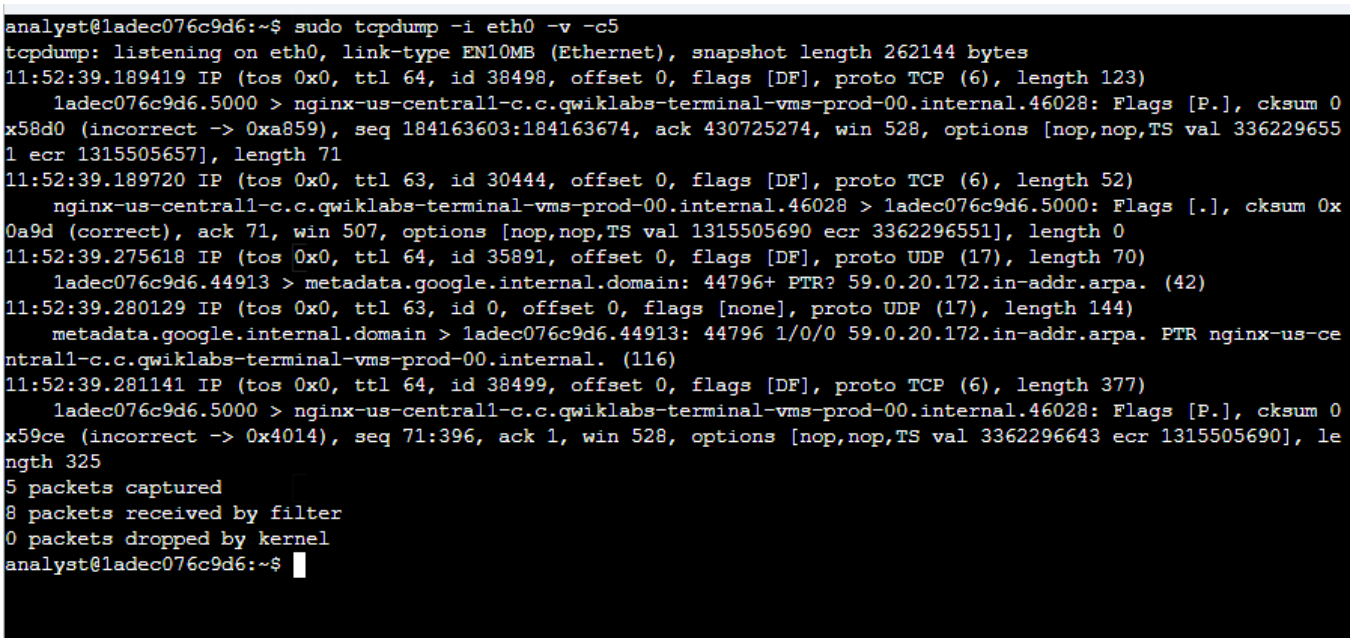
Command breakdown

Option	Purpose
`sudo`	Runs the command with elevated privileges.
`-i eth0`	Selects the `eth0` network interface.
`-v`	Displays more detailed packet information.
`-c5`	Stops after capturing five packets.

The output can include timestamps, IP information, Time to Live (TTL), protocol type, packet length, source and destination ports, TCP flags, and other packet details.

The exact output depends on the traffic present when the command runs.

Key learning: Verbose packet output helps analysts examine communication details beyond the basic source and destination addresses.



Task 3: Capture Network Traffic to a PCAP File

In this task, I used `tcpdump` to capture traffic associated with port 80 and save it to a file.

Capture command

```
sudo tcpdump -i eth0 -nn -c9 port 80 -w capture.pcap &
```

Command breakdown

Option	Purpose
`-i eth0`	Captures traffic from the selected interface.
`-nn`	Prevents IP address and port number name resolution.
`-c9`	Stops after capturing nine packets.

`port 80`	Filters traffic associated with port 80.
`-w capture.pcap`	Writes captured packets to a PCAP file.
`&`	Runs the command in the background.

After starting the capture, I used `curl` to generate web traffic.

Generate traffic

```
curl opensource.google.com
```

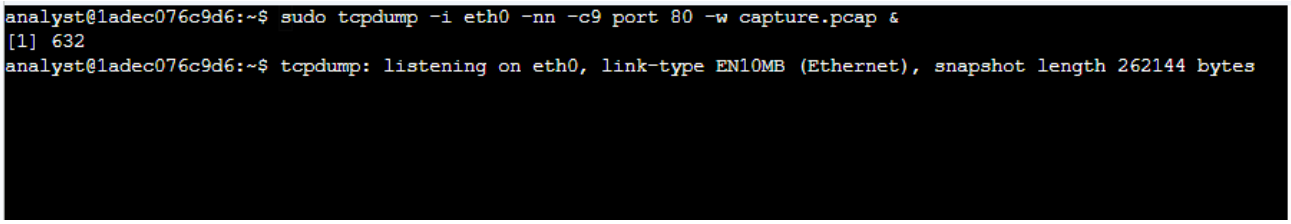
I then checked whether the capture file existed.

Verify the file

```
ls -l capture.pcap
```

The file listing can confirm that the PCAP file was created. The captured packet count and file size depend on the traffic generated and recorded during the lab.

Key learning: Saving packets to a PCAP file allows analysts to examine captured traffic after the capture process has finished.



Task 4: Filter and Inspect Captured Packet Data

After saving the capture, I used `tcpdump` to read the packet headers from the PCAP file.

Read packet headers

```
sudo tcpdump -nn -r capture.pcap -v
```

Command breakdown

- `-nn`: Disables IP address and port name resolution.
- `-r capture.pcap`: Reads packets from the saved capture file.
- `-v`: Displays detailed packet information.

This output can show source and destination IP addresses, ports, protocol information, TCP flags, packet lengths, and other header fields.

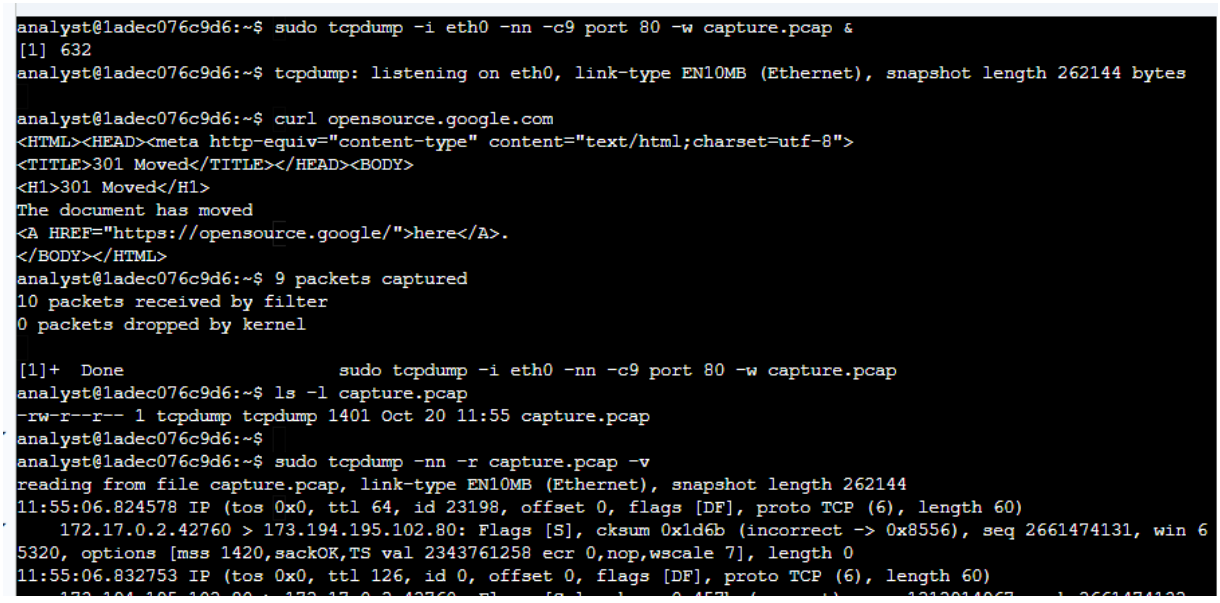
I then examined the packet contents in hexadecimal and ASCII formats.

Display hexadecimal and ASCII data

```
sudo tcpdump -nn -r capture.pcap -X
```

The `-X` option displays packet data in hexadecimal alongside its ASCII representation where applicable. This can help analysts recognize readable text, protocol information, or patterns in captured traffic.

Key learning: The `-r` option supports offline analysis of saved captures, while `-X` provides a closer view of packet contents.



```
analyst@1adec076c9d6:~$ sudo tcpdump -nn -r capture.pcap -X
reading from file capture.pcap, link-type EN10MB (Ethernet), snapshot length 262144
11:55:06.824578 IP 172.17.0.2.42760 > 173.194.195.102.80: Flags [S], seq 2661474131, win 65320, options [mss 14
20,sackOK,TS val 2343761258 ecr 0,nop,wscale 7], length 0
        0x0000:  4500 003c 5a9e 4000 4006 c2e1 ac11 0002  E..<Z.@.@.......
        0x0010:  adc2 c366 a708 0050 9ea2 df53 0000 0000  ...f...P...S....
        0x0020:  a002 ff28 1d6b 0000 0204 058c 0402 080a  ...(.k..........
        0x0030:  8bb2 f56a 0000 0000 0103 0307            ...j........
11:55:06.832753 IP 173.194.195.102.80 > 172.17.0.2.42760: Flags [S.], seq 1212914967, ack 2661474132, win 65535
, options [mss 1420,sackOK,TS val 32397217 ecr 2343761258,nop,wscale 8], length 0
        0x0000:  4500 003c 0000 4000 7e06 df7f adc2 c366  E..<..@.~......f
        0x0010:  ac11 0002 0050 a708 484b 9d17 9ea2 df54  .....P..HK.....T
        0x0020:  a012 ffff 457b 0000 0204 058c 0402 080a  ....E{..........
        0x0030:  01ee 57a1 8bb2 f56a 0103 0308            ..W....j....
11:55:06.832780 IP 172.17.0.2.42760 > 173.194.195.102.80: Flags [.], ack 1, win 511, options [nop,nop,TS val 23
43761266 ecr 32397217], length 0
        0x0000:  4500 0034 5a9f 4000 4006 c2e8 ac11 0002  E..4Z.@.@.......
        0x0010:  adc2 c366 a708 0050 9ea2 df54 484b 9d18  ...f...P...THK..
        0x0020:  8010 01ff 1d63 0000 0101 080a 8bb2 f572  .....c.........r
        0x0030:  01ee 57a1                                ..W.
```

Test Your Understanding

The lab also covered the following command-line concepts.

1. How can three packets be captured from any interface with verbose output?

```
sudo tcpdump -c3 -i any -v
```

2. What does the `-i` option specify?

It specifies the network interface to monitor.

3. What does the `-v` option do?

It enables verbose output, providing additional packet details.

4. Which command lists the available packet capture interfaces?

```
sudo tcpdump -D
```

Key Findings

This lab demonstrated the basic workflow for packet capture and analysis using `tcpdump`.

- Network interfaces can be identified with `ifconfig` and `tcpdump -D`.
- Live packets can be inspected with `tcpdump -i eth0 -v -c5`.
- Traffic can be filtered and saved to a PCAP file.
- Saved captures can be read with the `-r` option.
- The `-nn` option prevents address and port name resolution.
- The `-X` option displays packet data in hexadecimal and ASCII formats.

Key Learning

I learned how to use `tcpdump` to inspect network traffic from a Linux terminal, save selected packets, and analyze a PCAP file.

The activity also helped me understand how command-line options control packet capture and output. These skills provide a foundation for further learning in network security, network troubleshooting, and security operations center (SOC) investigations.

Conclusion

Packet capture and analysis are important parts of network security. `tcpdump` provides a practical way to monitor network interfaces, filter traffic, save packets, and inspect captured data.

The skills practiced in this lab can support future investigations involving suspicious network communications and provide a f